matthew karsten.

2026

Adyen Developer Relations: making the platform agent-ready in 90 days

Adyen

DEVELOPER RELATIONS ENGINEER APPLICATION

14-09-2026 matthewkarstenconnects@gmail.com Matthew Karsten

About me

Developer Relations engineer who writes the docs, builds the sample apps and ships the tooling developers actually run.



At ZetaChain I was the Developer Relations Engineer for a platform whose ecosystem grew to 10,000+ active developers. I owned the documentation, SDKs, starter kits and reference apps, and rewrote guides straight from the questions that kept recurring in Discord and Telegram.

I have shipped 15+ production MCP servers, including a 73-tool Hedera server and a Solana server used by 1,000+ developers; three published technical books; talks at EthCC 2025, ETH NYC and ETH Denver. Claude Certified Architect (Anthropic, 2026). B.F.A., Point Park University.

Our Mission:
Make the first hour with an API the best hour, and make the docs something you can test.

Why Matthew Karsten?

- I build the thing before I write about it: the audit and MCP server in this proposal were built against Adyen's live surfaces first.
- OpenAPI-native: the generated reference and the example validator read CheckoutService-v72.json, so they stay correct when the spec changes.
- Friction becomes a number, not an anecdote: 847 non-parsing snippets is a count CI can hold at zero.
- Agent-first: every page and snippet gets tested the way an agent reads it, as Markdown over HTTP.
- Relocating to San Francisco; office-first suits how I work.

Re: what I would do in the first 90 days

The posting sets an explicit quality bar: developer surfaces that are legible and reliable for agents as well as people, and content kept current because stale content misleads both.

I measured that bar instead of describing it. The tool and report are at github.com/ExpertVagabond/adyen-agent-ready; findings next page, then the plan.

> Ship a measurable improvement to Adyen's agent-readiness in 90 days: every advertised Markdown page fetchable by an agent, every snippet parseable, zero Checkout version drift, the API reference in llms.txt, and a monthly report to Product with numbers instead of adjectives.

What the audit found on docs.adyen.com

Measured on 2026-09-14 against docs.adyen.com/llms.txt (2,524 pages), llms-full.txt, sitemap.xml, robots.txt, Adyen/adyen-openapi CheckoutService-v72.json and the @adyen/mcp README. Full report with URLs: github.com/ExpertVagabond/adyen-agent-ready/blob/main/AUDIT.md

The posture is right. robots.txt explicitly allows GPTBot, ClaudeBot, PerplexityBot and three more; every page has a `.md` twin with `last_modified` in its frontmatter; llms.txt exists; an official MCP server is published. Very few payment platforms have gone this far. The gaps are in what an agent finds once it is through the door.

The edge blocks the agents the robots file invites in

- After roughly 1,000 Markdown fetches, docs.adyen.com answers a plain HTTP client with HTTP 429, `cf-mitigated: challenge` and a Cloudflare "Just a moment" HTML body on `.md` URLs. No `Retry-After`. An agent cannot execute the challenge and gets HTML where it expected Markdown.
- llms-full.txt is 51.1 MB, about 13.4M tokens. No model can load the file llms.txt points to as "the full corpus"; agents that try truncate silently to the Account section.

The reference agents are told to consult is unreachable

- llms.txt lists 0 API reference pages. The guides link to the API Explorer 11,370 times, and the Explorer is a JavaScript application with no Markdown rendering (`.md` returns 404).
- The official @adyen/mcp covers 8 of 29 Checkout operations; POST /payments and /payments/details, the Advanced flow, are not among them, and its README pins Checkout v71 while the spec and Node SDK are at v72.

What the audit found, continued

Examples, freshness and reach. Every count links to page URLs in AUDIT.md.

Examples agents copy do not run as written

- 847 of 7,870 JSON/JS/TS/Python snippets do not parse even as fragments (299 JSON examples contain `...`, 59 have trailing commas); 526 more parse only after dedenting or wrapping.
- Docs-template markup leaks into the Markdown: `{hint: ...}{/hint}` on 115 pages, 66 times inside JSON examples, including as a JSON key (`"{hint: ...}shopperReference{/hint}"` on five Klarna pages).
- 245 pages mix Checkout API versions in their own examples. The Sessions flow guide alone calls v70 twelve times, v71 six times and v72 twice. 67 pages never mention v72 at all.
- 9 request examples omit a field the v72 schema marks required (`returnUrl` on /payments); 270 code fences carry no language tag.

Freshness and reach

- 801 of 2,523 pages declare a `last_modified` more than three years old; 172 of those are in the integration sections and contain code.
- 6 pages exceed 100k tokens each because every language tab is inlined into one Markdown document (advanced-flow.md is 1.06 MB, 857 code blocks).
- 57,539 in-site links are relative (`/payment-methods`), so an agent reading a fetched page must guess the host and the `.md` suffix; 396 distinct targets are not in llms.txt. 42 sitemap pages are missing from llms.txt.

The plan: three 30-day milestones

Prepared for:
Adyen,
Each milestone ends with something shipped, something measured, and something reported. Nothing below needs a reorg or a budget.

(a) **Days 1 to 30: open the door and fix the seams.** With the web team, exempt `/*.md`, `/llms.txt` and `/llms-full.txt` from the Cloudflare managed challenge, or return a real 429 with `Retry-After`. Land the audit as a CI job on the docs repo: link liveness, snippet lint, version drift, template leaks, sitemap parity. Fix the `{hint:}` leak in the Markdown export. Template the Checkout version into every snippet from one constant.

- Agent crawl of all 2,524 pages: 0 challenge responses
- Template leaks: 115 pages to 0
- Version drift: 245 mixed pages to 0
- Snippet lint on every docs PR

(b) **Days 31 to 60: a reference and examples agents can read.** Generate one Markdown page per Checkout operation from the OpenAPI spec (the repo's `llms-checkout.txt` is the prototype) and list them under `## API reference` in llms.txt. Split llms-full.txt per product. Serve one `.md` per language tab. Replace `...` in JSON examples with complete bodies; validate every request example against its endpoint schema in CI.

- API reference: 29 Checkout operations in llms.txt, then every service
- Snippet pass rate: 89% to 99%+
- Largest page under 40k tokens per language
- Request examples: 0 unknown or missing required fields

(c) **Days 61 to 90: the loop and the agent bar.** Extend the official MCP server from the spec so coverage and version move together (/payments, /payments/details, test cards). Build an agent task-success benchmark: 20 real integration questions, run monthly against the docs with a fixed model, scored on whether the TEST API accepts the resulting request. Publish the monthly developer-experience report to Product and Engineering by page URL and endpoint.

- Agent task-success rate: baseline in month one, 90% by day 90
- Official MCP: 29 of 29 Checkout operations
- Monthly DX report, URLs not adjectives
- 172 stale integration pages reviewed or redirected

Scoped to public surfaces; inside Adyen the priorities will shift with support data, the method will not.

Already built: the audit (2,524 pages, 19,084 code blocks, 7,870 snippets linted, 17 findings with URLs), a Markdown reference for all 29 Checkout v72 operations, a 92-card test catalogue, and an 8-tool MCP server on the official Node SDK with 31 passing tests.

Scope:

`github.com/ExpertVagabond/adyen-agent-ready (MIT, TypeScript)`

Metrics I would report

One dashboard, reported monthly to Product, Engineering and the DevRel team. Every number below is computable from artifacts Adyen already publishes; none needs a survey. The first column is what the audit measured on 2026-09-14.

ITEM	INCLUDES	FEE
Agent fetch success	Share of llms.txt URLs an identified non-browser client can fetch as Markdown in one crawl	`Now 40% at 12 workers. Target 100%`
Snippet pass rate	Fenced JSON/JS/TS/Python blocks that parse as written	`Now 89.2%. Target 99%+`
Version drift	Pages mixing Checkout versions, or never naming the current one	`Now 245 and 67. Target 0`
Reference in llms.txt	Checkout operations with a Markdown reference page listed in llms.txt	`Now 0 of 29. Target 29`
Template leaks	Pages containing {hint:} or other unrendered docs markup	`Now 115. Target 0`
Agent task success	Of 20 real integration questions, the share where an agent reading only the docs produces a request the TEST API accepts	`Baseline month 1. Target 90%`
Stale integration pages	Integration pages with code and last_modified over 3 years old, unreviewed	`Now 172. Target 0`

Counts from audit/report.json in the adyen-agent-ready repo, generated against CheckoutService-v72.json; `npm run audit` refreshes every number. Docs quality is usually argued about; a drift count goes to zero and stays there because CI blocks the regression.

Timeline

Days 1 to 30

Open the door, fix the seams

Days 31 to 60

Reference and examples agents can read

Days 61 to 90

The loop and the agent benchmark

How I would work

Weeks 1 to 2: Land the audit as a CI job on the docs repo (link liveness, snippet lint, version drift, template leaks) and hand Product the first report with URLs. Read every support macro, forum thread and docs-feedback submission from the last 90 days; tag by endpoint and failure mode.

Weekly: One docs PR batch driven by the tags, one sample app or snippet-set improvement, one number reported: drift count, snippet pass rate, agent task-success rate on the top 20 integration questions.

Monthly: Publish the developer-experience report to Product and Engineering: what developers and agents asked, what shipped, what is still open, with page URLs and endpoint names, not adjectives.

Always: Every guide starts from a repo with tests. Every snippet is linted in CI. Nothing is published that I have not run against the TEST environment that day, and nothing is published that an agent cannot fetch as Markdown.